

FlutterTap

Relatório de Desenvolvimento

Módulo Zygisk (Magisk / KernelSU / APatch) para interceptação de tráfego de apps Flutter

Desenvolvido por Eduardo Lopes

Data: 26 de julho de 2026

1. Resumo executivo

Este projeto porta a lógica do script Frida `flutter+burp.js` — que contorna a verificação de certificado TLS (SSL pinning) do BoringSSL usado pelo engine do Flutter e redireciona o tráfego de rede do app para um proxy configurável — para um **módulo Zygisk nativo**, compatível simultaneamente com Magisk, KernelSU (via Zygisk/Zygisk Next) e APatch. Diferente do script original, que depende de uma sessão manual do Frida (via USB, com `frida-server` rodando), o módulo funciona de forma persistente e automática, sem qualquer ferramenta externa conectada.

Junto ao módulo, foi desenvolvido um **aplicativo gerenciador** (Jetpack Compose) que permite configurar, direto no aparelho, o IP/porta do proxy e quais apps instalados devem ser interceptados — exatamente como a maioria dos módulos Magisk/KernelSU do mercado (ex: seleção de apps por checkbox).

2. O script original

O `flutter+burp.js` é uma técnica pública e amplamente usada na comunidade de pentest mobile (baseada em pesquisa da NVISO sobre o engine do Flutter), que funciona assim:

1. Localiza `libflutter.so` carregada no processo do app.
2. Faz o parsing manual do cabeçalho ELF (segmentos `PT_LOAD` / `PT_GNU_RELRO`) para delimitar as regiões de memória onde procurar.
3. Escaneia a memória procurando as strings `"ssl_client"` e `"Socket_CreateConnect"`, usadas como pontos de referência.
4. A partir dessas strings, localiza (via padrões de instrução `adrp` / `add` no ARM64, ou `lea` relativo a RIP no x64) o endereço real das funções internas `verify_cert_chain` (verificação do certificado) e `GetSockAddr` (montagem do endereço de destino da conexão).
5. Instala 3 hooks: um em `GetSockAddr` para capturar o ponteiro do `sockaddr`; um em `socket()` para sobrescrever IP/porta desse `sockaddr` com os valores do proxy configurado; e um em `verify_cert_chain` para forçar o retorno "certificado válido".

Nenhuma dessas três funções é exportada/pública — por isso o script precisa "descobrir" os endereços em tempo real a cada execução, em vez de simplesmente chamar uma função pelo nome.

3. Arquitetura da solução

3.1 Módulo nativo (C++, Zygisk)

Lógica do script original	Equivalente no módulo nativo
<code>findAppId()</code>	<code>AppSpecializeArgs::nice_name</code> , lido em <code>preAppSpecialize</code>
<code>BURP_PROXY_IP</code> / <code>PORT</code> fixos no código	<code>/data/adb/fluttertap/config.json</code> , editável pelo app gerenciador
<code>parseElf()</code>	<code>elf_utils.cpp</code>
<code>scanMemory()</code> (busca por padrão com curingas)	<code>mem_scan.cpp</code>
<code>Instruction.parse</code> (Capstone por trás do Frida)	Capstone, linkado diretamente (<code>addr_resolver.cpp</code>)
Hook em <code>GetSockAddr</code> (onEnter)	<code>DobbyInstrument</code>
Hook em <code>socket()</code> (sobrescreve <code>sockaddr</code>)	<code>DobbyHook</code> na função global da libc
Hook em <code>verify_cert_chain</code> (onLeave)	<code>DobbyHook</code> com wrapper que força o retorno

A parte de resolução de endereço (ARM64 e x64) foi portada **instrução por instrução**, incluindo peculiaridades do script original (explicadas na seção 4), usando a mesma biblioteca de desmontagem que o próprio Frida usa por baixo dos panos (**Capstone**), em vez de decodificar manualmente os opcodes — isso elimina uma classe inteira de bugs sutis que uma reimplementação "à mão" correria o risco de introduzir.

3.2 Sistema de configuração compartilhada

O arquivo `/data/adb/fluttertap/config.json` guarda IP/porta do proxy, se a interceptação está ativa e a lista de pacotes-alvo. Ele fica *fora* da pasta do módulo (`/data/adb/modules/fluttertap`), para sobreviver a atualizações/reinstalações do módulo. Como o processo do zygote pode não ter permissão SELinux para ler arquivos em `/data/adb` diretamente, a leitura é feita através do **processo companion** do Zygisk (que roda como root irrestrito), via um protocolo simples sobre socket Unix.

3.3 App gerenciador (Jetpack Compose)

- Lista os apps instalados no aparelho e permite marcar quais devem ser interceptados (com busca).
- Campos para IP/porta do proxy, com validação, e um interruptor geral de ativação.
- Detecta root e o backend ativo (Magisk/KernelSU/APatch) e se o módulo está instalado — apenas para exibição, já que os três implementam a mesma API Zygisk.
- Grava a configuração via shell root (`libsu`), codificada em base64 para não precisar escapar caracteres especiais.
- 3 idiomas: Português (Brasil), Inglês e Chinês, trocáveis direto no app.
- Tema claro/escuro sempre acompanhando o tema do sistema (Compose + `values-night/`), corrigindo o problema relatado de telas com fundo branco mesmo com o celular em modo escuro.

4. Peculiaridades do script preservadas de propósito

Para o módulo se comportar **exatamente** como o script original, algumas particularidades — que poderiam parecer "erros" à primeira vista — foram mantidas intencionalmente:

- **Sem suporte a 32 bits.** O script original só trata `Process.arch == 'arm64'` ou `'x64'` — não existe um caminho para ARM32/x86 32 bits nem no original. Por isso o módulo também só é compilado para `arm64-v8a` (aparelhos reais) e `x86_64` (emuladores).
- **"a última correspondência vale".** O `Memory.scan` do script chama o callback para cada ocorrência encontrada, sempre sobrescrevendo a mesma variável — ou seja, se um padrão aparecer mais de uma vez, vale o *último* encontrado, não o primeiro. A busca de memória do módulo nativo reproduz esse mesmo comportamento.
- **Varredura byte a byte no x64.** Os trechos de "rastrear para trás" no x64 avançam 1 byte por vez tentando decodificar uma instrução válida a cada posição (em vez de avançar pelo tamanho real da instrução anterior). Isso foi copiado exatamente assim, pois é o que foi validado empiricamente contra builds reais do BoringSSL/Flutter.
- **Caso especial do app da Alibaba.** O script trata separadamente o padrão de bytes do app `com.alibaba.intl.android.apps.poseidon`, que difere dos demais — mantido no módulo.

5. Melhorias deliberadas (sem mudar o comportamento observável)

O script original foi feito para uma sessão curta e supervisionada por um humano. O módulo roda sozinho, durante toda a vida do app, então alguns pontos foram reforçados sem alterar o que acontece na rede:

- **Captura do sockaddr por thread** (em vez de uma única variável global), evitando condição de corrida entre conexões simultâneas.
- **Proteção contra SIGSEGV/SIGBUS** em volta de toda leitura de memória "heurística" — se um endereço calculado acabar sendo inválido, o módulo registra um aviso e desiste daquele trecho, em vez de derrubar o app do usuário.
- **Ocupação zero em apps não selecionados.** Antes de fazer qualquer trabalho, o módulo verifica se o processo atual está na lista de apps escolhidos; se não estiver, ele se descarrega da memória imediatamente (`DLCLOSE_MODULE_LIBRARY`).
- **Hook do `socket()` via Dobby direto**, e não pela API oficial de PLT hook do Zygisk — porque essa API só é válida até o fim do `postAppSpecialize`, e `libflutter.so` normalmente só carrega bem depois disso, durante a inicialização normal do app.

6. Desafios encontrados durante o desenvolvimento

Esta seção documenta os problemas reais enfrentados ao compilar o projeto pela primeira vez, e como foram resolvidos — útil para quem for dar manutenção no projeto depois.

6.1 Licença restritiva do Zygisk Next

O plano inicial era usar o header da API Zygisk a partir do repositório `ZygiskNext`, mas esse projeto mudou de licença e agora proíbe explicitamente copiar/extrair qualquer trecho de código dele. A solução foi usar a fonte oficial e correta: o repositório `topjohnwu/zygisk-module-sample`, mantido pelo próprio criador do Magisk, cujo header é licenciado em 0BSD (equivalente a domínio público) exatamente para uso livre por desenvolvedores de módulos.

6.2 Repositório Dobby com o backend Linux/Android quebrado

O commit mais recente do Dobby (biblioteca de hooking inline usada para os hooks internos) não compila para Android: há uma inconsistência real no próprio código-fonte deles entre o nome de um campo de struct (`RuntimeModule.base` vs. o uso de `.load_address` em vários arquivos) e entre um campo e um método (`MemRange.start` vs. `start()`). Isso foi confirmado de forma independente: o workflow de CI do próprio projeto Dobby define um job que deveria publicar um pacote pré-compilado para Android, mas esse artefato simplesmente não aparece nos releases — sinal de que o job falha silenciosamente há tempos.

A solução foi fixar o submódulo Git do Dobby em um commit anterior (`e9fe7fb`, de abril de 2023), o último antes da reestruturação que introduziu essa inconsistência, e aplicar dois pequenos ajustes locais por cima:

1. Uma função (`Logger::Shared()`) definida em um header estava sem a palavra-chave `inline`, causando erro de "símbolo duplicado" ao linkar.
2. O alvo de biblioteca compartilhada `dobby` (que não é usado pelo projeto — só a versão estática é) foi desativado apenas para Android, já que ele também falha ao linkar pelo mesmo motivo.

6.3 APIs experimentais do Jetpack Compose Material 3

Durante a build do app gerenciador, o componente `ExposedDropDownMenuBox` (usado inicialmente no seletor de idioma) e o `TopAppBar` básico se mostraram marcados como API experimental nesta versão do Compose, quebrando a build por padrão. O seletor de idioma foi reescrito com um `DropDownMenu`

simples (API estável), e foi adicionado um opt-in explícito (`@OptIn(ExperimentalMaterial3Api::class)`) para o restante.

6.4 Ambiente de build

A máquina tinha o Java 26 como padrão do sistema, versão recente demais para o Gradle 8.11.1 rodar. A solução foi apontar o `JAVA_HOME` para o JDK 21 que já vem embutido no próprio Android Studio instalado, evitando precisar baixar/installar outro JDK à parte.

6.5 Compatibilidade com builds mais novos do engine Flutter (app de demonstração)

Para conseguir capturas de tela e evidências sem usar apps de clientes, foi selecionado o **DVFA (Damn Vulnerable Flutter App)** — um app de treinamento open-source (MIT), mapeado nas 10 vulnerabilidades do OWASP Mobile Top 10, com APK pronto na aba Releases do GitHub (`github.com/Schmiemandev/dvfa`). O APK oficial não declarava a permissão `android.permission.INTERNET` (confirmado via `aapt dump badging`), então foi recompilado localmente (decompilado com `apktool` , permissão adicionada, reempacotado e assinado com a chave de debug) só para viabilizar o teste — sem alterar nenhuma outra funcionalidade do app.

Ao testar contra esse app, o módulo falhou em resolver o endereço de `verify_cert_chain` . A investigação revelou **três incompatibilidades reais** entre a lógica original do script e como esse binário específico (compilado com uma toolchain mais nova) é estruturado — nenhuma delas é um bug do FlutterTap em si, e sim uma limitação da técnica original que também afetaria o script Frida:

1. **Biblioteca mapeada direto do APK.** Builds recentes do Android/AGP usam `extractNativeLibs="false"` por padrão, então a `libflutter.so` não vira um arquivo próprio — fica mapeada por offset dentro do `base.apk` . A busca original (por caminho de arquivo em `/proc/self/maps`) não encontra nada nesse caso. Corrigido trocando para `dl_iterate_phdr` , que o bionic reporta corretamente nos dois casos.
2. **Segmentos de rodata e texto combinados.** Esse binário específico funde os segmentos ELF de rodata e de código executável em um único `PT_LOAD` (comportamento padrão de versões mais novas do linker lld), enquanto a lógica original assume que são dois segmentos separados. Corrigido identificando "o segmento executável" pela flag `PF_X` em vez de assumir qual seria o segundo `PT_LOAD` .
3. **Padrão de bytes preso a um registrador específico.** O padrão de bytes original do script para achar o par de instruções `adrp / add` (e o equivalente x64 com `lea`) na verdade fixa, nos "nibbles fixos" do padrão, o registrador específico que o compilador escolheu no binário onde o script foi originalmente desenvolvido. Um binário compilado com registradores diferentes para o mesmo cálculo simplesmente não bate com o padrão.

Corrigido buscando pela instrução em si (via Capstone) e validando o endereço calculado, independente de qual registrador foi usado — o que também tornou desnecessário o caso especial do app da Alibaba que o script original tinha (esse caso especial existia só por causa dessa mesma fragilidade).

Após as três correções, o módulo resolveu os endereços corretamente e instalou os hooks com sucesso no DVFA também.

6.6 Limitação conhecida: resolução do GetSockAddr em um build específico

Mesmo depois das três correções acima, o DVFA continuou sem interceptar o tráfego real de rede: os endereços "resolvem" e os hooks são instalados normalmente, mas a reescrita do `sockaddr` nunca dispara nas requisições reais do app (confirmado inclusive forçando uma conexão totalmente nova, alternando o Wi-Fi antes de tentar de novo — descartando a hipótese de reaproveitamento de uma conexão já existente).

Enquanto isso, o mecanismo já havia funcionado de ponta a ponta, com tráfego real capturado, contra um app de produção real de um cliente (testado de forma independente por Eduardo, no próprio celular — seção 10.4) — evidência de que o problema é específico desse build do DVFA, não da lógica do módulo em si.

A explicação mais provável: a resolução do `GetSockAddr` depende de contar quantas instruções `b1` existem a partir do `Socket_CreateConnect` e pegar o alvo da 2ª chamada (mesma lógica do script original). Nesse build específico do DVFA, esse mesmo procedimento provavelmente aponta para uma função diferente, que *parece* uma função de preenchimento de `sockaddr` (mesmo formato de prólogo, escreve a família do endereço no offset 0, zera um buffer do tamanho de um `sockaddr_storage`) mas não é de fato a função chamada nas conexões TCP reais desse app.

Essa investigação não foi levada adiante: seria necessário reverter todo o grafo de chamadas do `Socket_CreateConnect` desse build específico, sem garantia de solução rápida, e esse app de teste não é crítico para o projeto — a decisão (após conversa) foi não arriscar mexer na lógica de hook compartilhada por causa de um único app de teste.

Encaminhamento: o DVFA foi então descartado como app de demonstração e substituído por dois outros apps de treinamento públicos, ambos confirmados funcionando de ponta a ponta sem nenhuma mudança de código adicional — ver seção 10.

7. Estrutura do projeto

Modulo SukiSu/

- └─ module/ Módulo Zygisk nativo (Gradle + CMake)
 - └─ src/main/cpp/
 - └─ main.cpp Ponto de entrada Zygisk (pre/postAppSpecialize)
 - └─ module_config.* Schema e matching de pacotes-alvo
 - └─ companion.* Processo companion (leitura root do config)
 - └─ elf_utils.* Parsing do ELF de libflutter.so
 - └─ mem_scan.* Busca de padrões de bytes com curinga
 - └─ addr_resolver.* Resolução de endereços via Capstone
 - └─ hooks.* Instalação dos hooks via Dobby
 - └─ third_party/ Capstone e Dobby (submódulos git)
 - └─ template/ module.prop, customize.sh, uninstall.sh
 - └─ manager-app/ App gerenciador (Kotlin + Jetpack Compose)
 - └─ scripts/build_module_zip.sh Empacota o .zip flashável final
 - └─ docs/ Esta documentação

8. Compatibilidade

Item	Suporte
Versões do Android	Android 10 (API 29) até Android 17 (API 37, mais recente disponível no momento)
Arquiteturas	arm64-v8a (aparelhos reais) e x86_64 (emuladores)
Root	Magisk (Zygisk nativo), KernelSU (com Zygisk/Zygisk Next), APatch
Idiomas do app	Português (Brasil), Inglês, Chinês

9. Aviso legal e ético

Esta ferramenta contorna a verificação de certificado TLS de apps Flutter para fins de **análise de tráfego** (pentest mobile, engenharia reversa, pesquisa de segurança). Ela deve ser usada apenas em aplicativos que você tem autorização para testar (seus próprios apps, ou em um engajamento de teste de segurança autorizado). O módulo não tem nenhum alvo específico embutido — a seleção de quais apps interceptar é sempre manual, feita pelo usuário através do app gerenciador.

10. Validação em dispositivo real

O módulo foi testado em um aparelho físico: um **OnePlus 5, Android 10, arquitetura arm64-v8a**, com root via Magisk (fork "Kitsune Magisk" v27.2) e Zygisk fornecido pelo módulo **NeoZygisk** (já que esse fork não usa o Zygisk nativo). O Burp Suite já estava rodando na máquina do desenvolvedor, escutando em `192.168.15.13:8083`, na mesma rede Wi-Fi do aparelho.

10.1 Apps de demonstração escolhidos

Para ter evidências reais sem usar nenhum app de cliente nas capturas, foram usados dois apps de treinamento de segurança **públicos e oficiais**, ambos em Flutter:

App	Autor / licença	Por que serve como demonstração
Ostorlab Insecure Android App <code>github.com/Ostorlab/ostorlab_insecure_android_app</code>	Ostorlab (empresa de segurança mobile), Apache-2.0	Módulo Flutter com um teste dedicado de tráfego TLS (<code>TLSTraffic</code>), fazendo uma requisição HTTPS real para <code>ostorlab.co</code>
VulnApp <code>github.com/anasachoury/VulnApp</code>	anasachoury	App Flutter simples mapeado no OWASP Mobile Top 10, com uma chamada de rede real (HTTP, para <code>httpbin.org</code>)

Nenhum dos dois tinha um APK pronto (nem em Releases, nem em artefatos de CI) — foi necessário instalar o Flutter SDK e compilar os dois a partir do código-fonte. Ambos também estavam com a permissão `android.permission.INTERNET` faltando no manifesto (o mesmo tipo de descuido encontrado antes no DVFA e no `burptestapp` — aparentemente comum nesse tipo de app de treinamento pequeno) e precisaram de pequenos ajustes de dependências desatualizadas (ver seção 10.5) para compilar com o toolchain atual. Nenhuma mudança foi feita na lógica de vulnerabilidade/teste dos apps em si.

10.2 Passos executados

1. Empacotamento do módulo (`scripts/build_module_zip.sh`) e instalação via `magisk --install-module` .

2. Compilação dos dois apps de demonstração a partir do código-fonte (Flutter SDK instalado localmente) e instalação via `adb install`.
3. Escrita do `config.json` apontando para o proxy real e para o pacote de cada app, um de cada vez.
4. Reinicialização do aparelho para o Zygisk carregar o módulo.
5. Abertura de cada app, interação com a funcionalidade que dispara a chamada de rede, e acompanhamento do log do módulo (`adb logcat`) e do histórico de proxy do Burp.

10.3 Resultado

Sucesso confirmado com tráfego real nos dois apps, sem nenhuma mudança de código entre eles. O log do módulo mostrou a sequência completa esperada em ambos: `libflutter.so` carregada → endereços de `verify_cert_chain` e `GetSockAddr` resolvidos corretamente via Capstone → hooks instalados → `sockaddr` reescrito a cada nova conexão para `192.168.15.13:8083`.

No **Ostorlab Insecure App**, ao acionar o teste `TLSTraffic`, o Burp capturou e decodificou de verdade uma requisição HTTPS para `https://ostorlab.co/` (resposta HTTP/2 200 OK completa, com o HTML da página) — **sem precisar instalar nenhum certificado no aparelho**, exatamente o comportamento esperado do bypass de verificação de certificado.

No **VulnApp**, ao tocar em "Call Insecure API" na tela de perfil, o Burp capturou a requisição HTTP para `http://httpbin.org/get` — confirmando que o redirecionamento de tráfego funciona independente de a conexão ser HTTPS (com bypass de certificado) ou HTTP simples.

Observação sobre o login do VulnApp: ao testar, não apareceu nenhuma requisição de rede no momento do login. Investigando o código-fonte (`lib/screens/login_screen.dart`), o motivo é que a autenticação é inteiramente local — usuário/senha (`admin / 1234`) estão fixos no próprio código Dart, e a única "espera" é um `Future.delayed` simulando um carregamento, sem nenhuma chamada de rede real. A única requisição de rede de todo o app é mesmo a do botão "Call Insecure API". Não é uma falha do módulo — simplesmente não existe requisição nenhuma para interceptar nesse ponto.

10.4 Validação independente contra um app real de produção

Em uma rodada de testes separada, Eduardo confirmou o módulo funcionando contra um app real de produção de um cliente seu, usando apenas o app gerenciador para selecionar o alvo e configurar o proxy, sem nenhum ajuste de código. Essa validação (feita de forma independente, em outro momento, por outra pessoa) é uma prova mais forte do que qualquer app de teste:

confirma que a abordagem genérica (mesma lógica de scan/resolução para qualquer app Flutter, nada hardcoded por aplicativo) funciona em produção.

Foi observado um comportamento onde a interceptação só passou a funcionar depois de reabrir o app gerenciador uma segunda vez. A explicação mais provável não é um bug no gerenciador (que só grava o `config.json`; ele não tem nenhuma ligação em tempo real com o processo nativo) e sim o comportamento normal do Zygisk: hooks só entram em processos *novos*. Se o app-alvo já estava rodando em segundo plano (comum em apps de loja/apps que o usuário mantém aberto), reabri-lo apenas traz o processo existente para frente, sem novo hook — é necessário forçar a parada do app-alvo (não do gerenciador) antes de reabrir para o hook valer, algo que já consta no guia de instalação (seção 11).

10.5 Compilando os apps de demonstração a partir do código-fonte

Como nenhum dos dois tinha APK pronto, foi necessário instalar o Flutter SDK e resolver algumas incompatibilidades de dependências antigas (ambos os projetos têm alguns anos e não foram atualizados para o toolchain atual). Nenhuma dessas mudanças afeta a funcionalidade de rede/TLS testada.

Passo	Detalhe
1. Instalar o Flutter SDK	Baixar o canal stable em <code>flutter.dev</code> , extrair em um caminho curto (ex: <code>C:\fdev\flutter</code>) — caminhos longos/aninhados no Windows causam erro de "nome de arquivo muito grande" durante o <code>pub get</code> .
2. <code>flutter config --android-sdk "<caminho do SDK>"</code>	Aponta o Flutter para o Android SDK já instalado (o mesmo usado pelo módulo nativo).
3. Adicionar <code>android.permission.INTERNET</code>	Faltava no manifesto dos dois apps — sem essa permissão nenhuma conexão de rede é possível, com ou sem o FlutterTap.
4. Remover dependências legadas incompatíveis	No Ostorlab: <code>flutter_android</code> (sem null-safety), <code>flutter_inappwebview</code> / <code>webview_flutter</code> (API de Proguard descontinuada), <code>local_auth</code> e <code>path_provider</code> (versões novas dependem do pacote <code>jni</code> , incompatível com a versão do Kotlin Gradle Plugin atual), e <code>android_intent_plus</code> (exige <code>compileSdk</code> mais novo que o do projeto). Todas essas dependências eram usadas só por testes de vulnerabilidade <i>não relacionados</i> a rede/TLS (biometria, webview, intents, etc.) — removidas junto com o código Dart que as usava, sem afetar o teste de TLS que interessa aqui.

5. `flutter build apk --debug`

Gera o APK em `build/app/outputs/flutter-apk/` (app normal) ou `build/host/outputs/apk/debug/` (caso de um "Flutter module", como o do Ostorlab, que gera um host de teste automaticamente).

10.6 Sobre o app de teste descartado (DVFA)

O **DVFA (Damn Vulnerable Flutter App)** foi o primeiro app de treinamento usado para tentar gerar evidências sem apps de cliente. Como documentado na seção 6.5/6.6, testar contra ele foi o que revelou as três incompatibilidades reais de builds mais novos do engine Flutter (e as correções aplicadas), mas o DVFA em si acabou sendo descartado como app de demonstração por causa de uma limitação específica daquele build (resolução do `GetSockAddr` para uma função aparentemente errada). Foi substituído pelos dois apps das seções 10.1–10.3, ambos confirmados funcionando sem ressalvas.

11. Guia de instalação manual (em um aparelho novo)

11.1 Arquivos necessários

Arquivo	Como gerar	O que é
FlutterTap- <versão>.zip	<code>./scripts/build_module_zip.sh</code>	Módulo flashável (Magisk/KernelSU/APatch)
manager-app-debug.apk (ou release)	<code>./gradlew :manager-app:assembleDebug</code>	App gerenciador

Esses dois arquivos são gerados a partir do código-fonte e não ficam versionados no Git — para distribuir prontos, o caminho padrão é anexá-los a uma *Release* do GitHub.

11.2 Passo a passo

1. **Ter root com Zygisk ativo.** Magisk, KernelSU ou APatch instalado; Zygisk ligado em Configurações → Zygisk (no Magisk) ou, se o Zygisk nativo não funcionar bem no seu fork/ROM, instalar e habilitar o módulo **Zygisk Next / NeoZygisk** antes de tudo. Reiniciar depois.
2. **Copiar o .zip do módulo para o aparelho.**
3. **Instalar o módulo** pelo app do Magisk/KernelSU/APatch → aba Módulos → "Instalar a partir do armazenamento" → selecionar o .zip. (Ou, via terminal root: `magisk --install-module /caminho/FlutterTap.zip`.)
4. **Reiniciar o aparelho** — obrigatório, o Zygisk só injeta em processos criados depois de carregado no boot.
5. **Instalar o app gerenciador** (.apk), permitindo "fontes desconhecidas" se necessário.
6. **Abrir o FlutterTap e conceder root** quando solicitado.
7. **Selecionar o(s) app(s)-alvo** e configurar IP/porta do proxy (o IP da máquina rodando o Burp/proxy, na mesma rede Wi-Fi do aparelho).
8. **Forçar parada e reabrir o app-alvo** — o hook só entra em processos novos.

12. Próximos passos

✓ Testar o módulo em um dispositivo real contra um app Flutter conhecido e validar a captura de tráfego no Burp Suite — **concluído** (seção 10), incluindo confirmação independente contra um app real de produção (seção 10.4).

✓ Ajustar a resolução de endereços para builds mais novos do engine Flutter/BoringSSL — **três correções aplicadas e validadas** (seção 6.5); um caso específico (`GetSockAddr` em um build particular, seção 6.6) ficou documentado como limitação conhecida em vez de arriscar mudar a lógica compartilhada sem necessidade.

3. Tirar capturas de tela reais do app gerenciador e dos dois apps de demonstração (Ostorlab Insecure App e VulnApp, já compilados e prontos no aparelho) para ilustrar o README/tutorial — pendente, a fazer na próxima sessão.
4. Publicar o projeto no GitHub: README completo, tutorial de instalação, capturas de tela, licença.
5. Preparar e enviar o módulo para a página oficial de módulos do Magisk ([Magisk-Modules-Repo](#)), seguindo os requisitos de submissão desse repositório.

— *Eduardo Lopes*